PortSwigger Lab - Insecure Direct Object References (IDOR)

Platform: PortSwigger Web Security Academy
Category: Access Control Vulnerabilities
Lab Link: https://portswigger.net/web-security/access-control/lab-insecure-direct-object-references

1. Objective

Understand how insecure direct object references can expose private application data when users can manipulate resource identifiers without proper authorization checks. Learn how predictable file references can allow unauthorized access to other users' chat transcripts and potentially lead to account compromise.

2. Key Concepts Learned

- **IDOR (Insecure Direct Object Reference):** A vulnerability that occurs when an application uses user-controlled references to access internal objects without properly verifying the user's permissions.
- **Direct Object References:** Identifiers used to access resources such as database records, files, user profiles, and chat transcripts.
- **Static File Exposure:** Occurs when sensitive files are stored on a server and can be retrieved directly through publicly accessible URLs.
- **Predictable Identifiers:** Sequential filenames or numeric IDs can make it easier to discover resources belonging to other users.
- **Horizontal Privilege Escalation:** Accessing another user's information or resources while operating at the same privilege level.
- **Sensitive Information Disclosure:** Exposure of private information, such as chat conversations, personal details, or credentials.
- **Credential Exposure:** Passwords or other authentication information disclosed through an application can potentially be used to access another account.
- **Broken Object-Level Authorization (BOLA):** An API security weakness in which an application does not properly enforce authorization when users request individual objects. This is closely related to IDOR.

3. Tools and Websites Covered

- **PortSwigger Web Security Academy:** Provided the intentionally vulnerable application and practical IDOR exercise.
- **Web Browser:** Used to interact with the application, access live chat, inspect transcript URLs, and test modified file references.
- **Live Chat Functionality:** Used to generate a chat transcript and examine how the application stores and retrieves conversation data.
- **View Transcript Feature:** Used to access the transcript through its static file URL.

Note: Burp Suite is not required for the documented solution to this lab. The vulnerability can be demonstrated directly through the browser.

4. Practical Work

- Opened the PortSwigger lab and accessed the vulnerable web application.
- Selected the Live Chat section and sent a message to generate a conversation transcript.
- Selected **View Transcript** to examine how the application provided access to the saved conversation.
- Inspected the transcript URL and observed that the application stored conversations as text files with filenames containing incrementing numbers.
- Modified the transcript filename to `1.txt` to access another stored conversation.
- Examined the retrieved transcript and identified a password belonging to the target user, `carlos`.
- Returned to the main application and used the discovered credentials to log in as `carlos`.
- Confirmed successful lab completion through the PortSwigger interface.

Useful URL Pattern

```
/files/<transcript-number>.txt
```

Example reference:

```
/files/1.txt
```

The actual transcript URL should be taken from the lab instance, as the directory structure may be specific to the environment.

5. Important Security Observations

- **Predictable filenames expose the attack surface.** Sequential transcript numbers can allow users to request files belonging to other users.
- **Static files still require access control.** A file does not become safe simply because it is stored separately from the application's database.
- **The main vulnerability is missing authorization.** The server retrieves a transcript without adequately checking whether the requesting user is permitted to access it.
- **Changing a filename can expose another user's data.** This is a simple example of object-reference manipulation.
- **Sensitive information disclosure can lead to account compromise.** Private chat records may contain credentials or other information that can be used to access accounts.
- **HTTPS does not prevent IDOR.** Encryption protects data in transit, but it does not correct missing authorization checks on the server.
- **A successful login does not justify the original access.** The security failure happens when the application exposes another user's private transcript without authorization.

6. Security Recommendations

- Enforce server-side ownership and authorization checks whenever a transcript or other private resource is requested.
- Avoid exposing sensitive files through directly accessible public URLs.
- Store private transcripts outside the public webroot and serve them through an authenticated application endpoint.
- Use unpredictable identifiers where appropriate, but never treat UUIDs or random filenames as a replacement for authorization.
- Ensure that users can access only their own conversations unless explicit permissions allow otherwise.
- Never store passwords, authentication tokens, or other sensitive credentials in chat transcripts.
- Apply secure data-retention policies and remove transcripts when they are no longer required.
- Test object-level access controls by attempting to access resources belonging to other users in an authorized testing environment.

7. Key Learnings

This lab helped me understand how a seemingly simple file-retrieval feature can introduce a serious access-control vulnerability. By examining the transcript URL, I learned how predictable filenames can expose other users' private conversations when the server does not enforce proper authorization.

The most important lesson was that **every object request must be checked against the identity and permissions of the requesting user**. Even when an application uses static files instead of database records, those files can still contain sensitive user information and must be protected.

I also learned how information disclosure can become a more serious security issue when exposed data contains authentication credentials. This demonstrated the connection between IDOR, horizontal privilege escalation, and potential account takeover.

Status: Complete